Fire Sentinel — Prototype 2 wiring diagram

Updated scope: BME280, ADS1115 (MQ-2 / MQ-7 / MQ-135 / UV), B35 flame sensor, INA219 power monitor, SSD1306 OLED, buzzer, green/yellow/red status LEDs, diffuse/acknowledge button + info button, 3.7V LiPo power. Mic dropped from scope. SD logging remains Phase 2.

GPIO allocation — full picture

Worth having this in one place now that pin budget is getting tight. Strapping pins (0, 3, 45, 46) and PSRAM-reserved pins (26-37 on this octal-PSRAM N16R8 module) are deliberately avoided throughout.

GPIO	Used for	Status
1	B35 flame sensor (digital)	used
2	info button	used
4,5,6,7,8,9,10,11,12,13,15,16,17,18	camera ribbon (fixed, internal)	used
14	Red LED (FIRE_CONFIRMED indicator)	used
19	diffuse/acknowledge button	used
20, 21	shared I2C bus — ADS1115, BME280, INA219, SSD1306 OLED	used
38, 39, 40	SD_MMC (Phase 2, reserved not yet wired)	reserved
41	Yellow LED (PRECURSOR/CONFIRMING indicator)	used
42	Buzzer	used
43, 44	UART0 Serial (via bridge chip — required, see hardware learnings)	used
47	Green LED (NORMAL/CONTROLLED_FLAME indicator)	used
48	onboard WS2812 LED	used

Power rail plan

Now four rails plus a battery input stage. Wire your breadboard's two power rail pairs like this:

Rail	Feeds	Source

5V	MQ-2, MQ-7, MQ-135 heater coils (VCC only)	Boost converter output (battery) or ESP32-S3 5V/VIN pin (USB bench testing)
3.3V	ADS1115, BME280, INA219, SSD1306, LEDs (via resistors), buzzer, flame sensor, UV sensor, button pull-up	ESP32-S3 3.3V pin
GND	everything — common ground, jumper both rail pairs' GND together	ESP32-S3 GND
BAT	system power source, monitored by INA219	3.7V LiPo → TP4056 → boost converter → 5V/VIN

> Keep the I2C bus on the 3.3V rail specifically, not 5V. Mixing 5V I2C devices with the ESP32's 3.3V-only GPIO without a level shifter can damage the pins. MQ sensors are the only things that actually need 5V — their heater coil needs it for accurate readings — and their output goes through a divider before it ever reaches ADS1115 anyway.

Component-by-component connections

Battery power path — 3.7V LiPo

A raw LiPo cannot connect directly to either the board's 5V or 3V3 pin. Full charge is 4.2V — that exceeds the ESP32-S3's absolute maximum input voltage (3.6V) if fed straight into 3V3, and it's below the ~4.5V minimum most boards' onboard regulators need if fed into VIN/5V as the battery discharges. You need a small conversion stage either way:

Stage	Component	Connects to
1. Charge + protect	TP4056 module (with built-in protection IC — most cheap ones include this)	BAT+/BAT- to the LiPo cell directly; IN+/IN- to a USB cable for charging
2. Step up	Small 3.7V→5V boost converter module (e.g. MT3608)	Input from TP4056's OUT+/OUT-; output feeds the 5V rail
3. Monitor	INA219 (see below)	Sits in series on the boost converter's output-positive line, before it reaches the 5V rail
4. Physical switch	Inline SPDT slide switch (not a GPIO)	Between TP4056 OUT+ and the boost converter input — lets you fully cut power without a microcontroller in the loop

Check your specific Freenove board for an onboard 2-pin JST "BAT" connector near the USB port first — if it has one with built-in charge management, you may be able to skip stages 1-2 entirely and connect the LiPo there directly. Nothing in the project notes so far confirms this board has one, so the external TP4056+boost path above is the safe default — verify against your actual board before assuming either way.

INA219 (I2C power/current monitor — address 0x40)

INA219 pin	Connects to
VCC	`3.3V rail` (logic power — separate from what it's measuring)
GND	`GND rail`
SCL	ESP32-S3 `GPIO21` (same I2C bus)
SDA	ESP32-S3 `GPIO20` (same I2C bus)
VIN+	boost converter output (the battery-side power path)
VIN-	continues on to the 5V rail — INA219 sits in-line, it doesn't tap off to the side

This is the one component that isn't a simple tap off the rail — current flows THROUGH it. VIN+ and VIN- are the two ends of the circuit it's measuring, not power-in/ground for itself.

SSD1306 OLED (128×64, I2C — address 0x3C)

OLED pin	Connects to
VCC	`3.3V rail`
GND	`GND rail`
SCL	ESP32-S3 `GPIO21` (same I2C bus)
SDA	ESP32-S3 `GPIO20` (same I2C bus)

Four I2C devices now share GPIO20/21 (ADS1115 0x48, BME280 0x76, INA219 0x40, OLED 0x3C) — all different addresses, no conflicts. If you ever see garbled I2C reads once everything's connected, it's almost always a wiring/pull-up issue from too many devices on one bus, not an address collision.

Status LEDs — green / yellow / red

LED	Anode (long leg)	Cathode (short leg)	Meaning
Green	220Ω resistor → ESP32-S3 `GPIO47`	`GND rail`	NORMAL / CONTROLLED_FLAME
Yellow	220Ω resistor → ESP32-S3 `GPIO41`	`GND rail`	PRECURSOR / CONFIRMING
Red	220Ω resistor → ESP32-S3 `GPIO14`	`GND rail`	FIRE_CONFIRMED

Standard pattern for each: GPIO → 220Ω resistor → LED anode (longer leg) → LED cathode (shorter leg) → GND. The resistor limits current to a safe ~10-15mA at 3.3V — don't skip it, driving an LED directly from a GPIO with no resistor shortens its life and can stress the pin. If board space genuinely runs out later, green is the one to keep — it's the only one with prior planning behind it — and yellow/red can drop back to dashboard-only indication.

Buzzer

Buzzer pin	Connects to
VCC (if a 3-pin driven module)	`3.3V rail`
GND	`GND rail`
I/O or + (signal)	ESP32-S3 `GPIO42`

If you have a bare passive piezo element (2 legs, no onboard driver) rather than a 3-pin module, it wires directly across GPIO42 and GND with no VCC connection — low enough current draw to drive straight from the GPIO. Either way, use `ledcWriteTone()`/`tone()`-style PWM output rather than a flat digitalWrite HIGH/LOW, so you can give each alert state a genuinely distinct pattern (e.g. a single short beep for PRECURSOR, a faster double-beep for CONFIRMING, a continuous tone for FIRE_CONFIRMED) instead of one generic buzz for everything.

Diffuse / acknowledge button

Button pin	Connects to
One leg	ESP32-S3 `GPIO19`
Other leg	`GND rail`

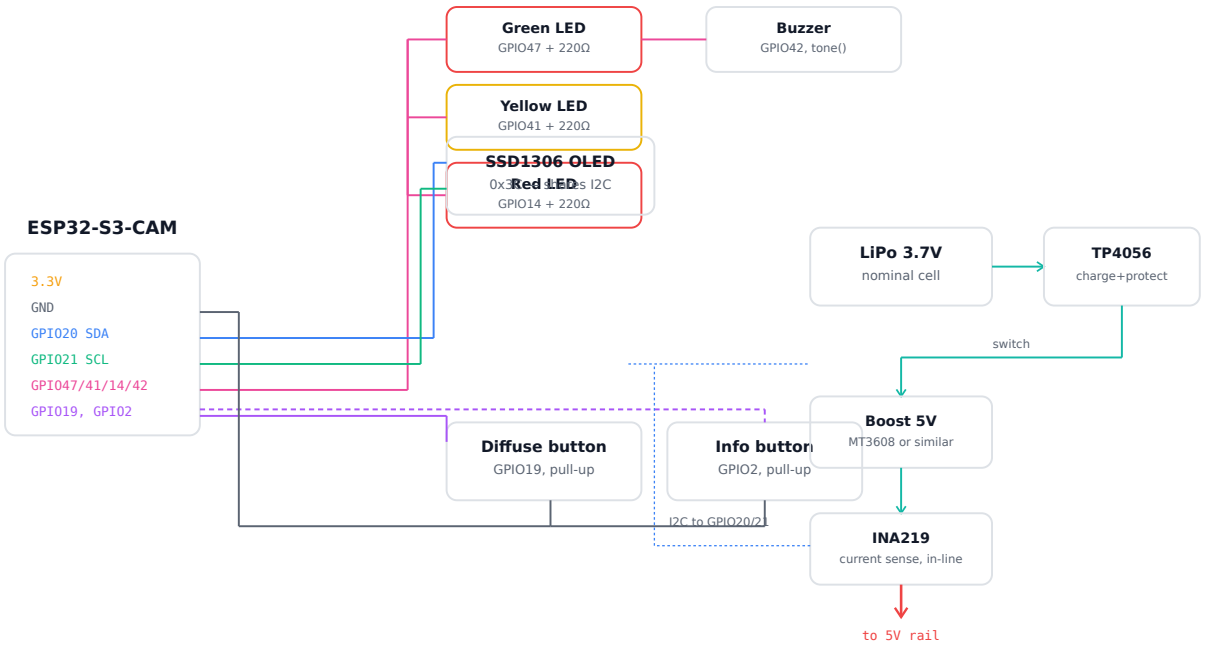
Wire with no external resistor — use the firmware's internal pull-up (`pinMode(19, INPUT_PULLUP)`) so the pin reads HIGH when unpressed and LOW when the button shorts it to GND. Purpose: lets someone silence the buzzer and acknowledge a false alarm (e.g. a burnt piece of toast triggering PRECURSOR) without needing the dashboard — press-to-diffuse, not an info/status button. No firmware logic for this exists yet; wiring it now just gets the physical pin ready.

Info button

Button pin	Connects to
One leg	ESP32-S3 `GPIO2`
Other leg	`GND rail`

Same wiring pattern as the diffuse button — internal pull-up (`pinMode(2, INPUT_PULLUP)`), no external resistor needed. Purpose: press to cycle the OLED through status screens on demand (battery %, current risk score, sensor readings, WiFi/backend connection status) rather than the OLED just showing one fixed view. This is the two-button setup from the original plan (diffuse + info) — both now have real pins, matching the panel-mount button hardware.

Schematic — display, power, LEDs, buzzer, button



The battery chain (left to right, top to bottom): LiPo → TP4056 → physical switch → boost converter → INA219 (in-line, measuring what flows through) → 5V rail. INA219 also reports back to the ESP32 over the same I2C bus as the OLED and other sensors. LEDs and buzzer are simple GPIO taps off the 3.3V/GND rails, shown here without every GND wire drawn individually — see the component tables above for the complete, exact connections.

Component-by-component connections

ADS1115 (I2C ADC — address 0x48)

ADS1115 pin	Connects to
VDD	`3.3V rail`
GND	`GND rail`
SCL	ESP32-S3 `GPIO21`
SDA	ESP32-S3 `GPIO20`
A0	MQ-2 divider output
A1	MQ-7 divider output
A2	MQ-135 divider output
A3	UV sensor AOUT (direct, no divider needed)
ADDR	leave unconnected (defaults to 0x48, matching config.h)

BME280 (I2C temp/humidity/pressure — address 0x76)

BME280 pin	Connects to
VCC	`3.3V rail`
GND	`GND rail`
SCL	ESP32-S3 `GPIO21` (same I2C bus as ADS1115)
SDA	ESP32-S3 `GPIO20` (same I2C bus as ADS1115)

MQ-2, MQ-7, MQ-135 (each identical wiring pattern)

Module pin	Connects to
VCC	`5V rail`
GND	`GND rail`
AOUT	top of its own 10kΩ/10kΩ divider (see below), **not** directly to ADS1115

Divider per sensor: AOUT → 10kΩ resistor → node → 10kΩ resistor → GND. Take the ADS1115 input wire from the *node* between the two resistors (not from AOUT directly). This halves the ~5V swing down to a safe ~2.5V max. You need three of these dividers — one per MQ sensor, 6 resistors total.

GUVA-S12D (UV sensor)

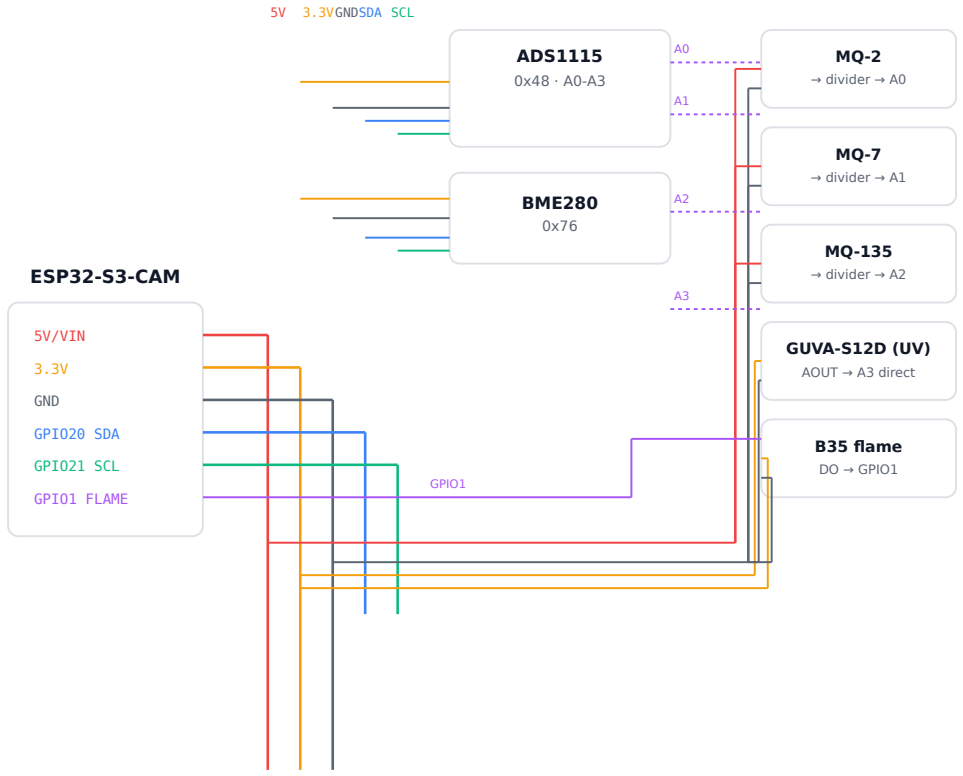
Module pin	Connects to
VCC	`3.3V rail`
GND	`GND rail`
AOUT	ADS1115 **A3** directly — output is low-voltage (~0-1V), no divider needed

B35 flame sensor (digital)

Module pin	Connects to
VCC	`3.3V rail` — keeps DO output at safe 3.3V logic level for the GPIO
GND	`GND rail`
DO	ESP32-S3 `GPIO1`

Verify your specific module: most LM393-comparator flame breakouts work fine at 3.3V, but check yours detects reliably at that voltage before trusting it — this was flagged as an open item from bring-up.

Schematic overview



Dashed purple = point-to-point analog signal (through its divider for the three MQ sensors) · solid lines = shared bus rails

Physical breadboard zone plan

This is now a lot to fit on one board. A full-size breadboard's two power rail pairs map onto 5V/3.3V — dedicate the top pair to 5V/GND, the bottom pair to 3.3V/GND, jumper the two GND lines together once.

Zone	Place here	Why
Left third	ADS1115, BME280, INA219, SSD1306 OLED	All four share the I2C bus — cluster them near where GPIO20/21 enter from the ESP32 to keep bus wiring short
Middle third	MQ-2, MQ-7, MQ-135 + their 6 divider resistors	Heaviest current draw — closest to the 5V rail entry point
Right third	UV sensor, flame sensor, LEDs + resistors, buzzer, diffuse button, info button	Lower pin count each, fills remaining space; group the three LEDs together so their resistor leads don't tangle with the gas sensor wiring in the middle zone

The battery chain (TP4056, switch, boost converter) doesn't need breadboard space at all if you use pre-assembled modules — they can sit off to the side or below the board, connected by just two wires (5V+ and GND) into the breadboard's power rail. The ESP32-S3-CAM board itself sits beside the breadboard, not plugged into it — jumper wires run from its header pins into the rails/columns.